Qué significa «le gusta»

Guía para responder al bloque C (señal objetivo) del recomendador de CPIA

Arnau Montull para Uriel Martínez

15 de octubre de 2026

Resumen

Esta guía reúne todo lo necesario para decidir cómo convertimos las notas de MovieLens en la señal con la que se entrena la Two-Tower: feedback explícito e implícito, pérdida contrastiva, negativos del lote, corrección logQ, umbrales, sesgo de cada usuario, pesos, métricas y el problema de la fuga de información. Cada concepto viene con un ejemplo y con las cifras que hemos medido en nuestra submuestra de desarrollo (3 154 155 ratings, 20 085 usuarios). Al final están las 14 preguntas (C1–C14) con sus opciones y mi recomendación (⋆), que *no* es una decisión. Los números salen de `python -m src.analisis.senal_objetivo` (`reports/senal_objetivo.md`) y la versión corta del planning está en `docs/planning_bloque_C.md`.

Índice

1. Dónde estamos y qué se decide aquí **3**
1.1. Lo ya decidido (bloques A y B) . . . 3
1.2. La arquitectura que prometimos en E1 . . . 3
1.3. Qué decide el bloque C . . . 3

2. Los datos que tenemos: notas explícitas **3**
2.1. Explícito frente a implícito . . . 3
2.2. Los datos no faltan al azar . . . 4
2.3. Cada usuario usa la escala a su manera . . . 4
2.4. Las medias estrellas no existían antes de 2003 . . . 4
2.5. Mucha gente lo puntúa todo el primer día . . . 4
2.6. Las populares gustan más (de verdad) . . . 5

3. Tres usuarios inventados **5**

4. Cómo aprende una Two-Tower **5**
4.1. Dos torres y un producto escalar . . . 5
4.2. Opción 1: regresión de la nota . . . 6
4.3. Opción 2: pérdida contrastiva (softmax) con negativos del lote . . . 6
4.4. Sesgo de muestreo y corrección logQ . . . 6
4.5. Falsos negativos . . . 7
4.6. Negativos duros . . . 7

5. De nota a positivo: umbrales **7**
5.1. Umbral absoluto, relativo o mixto . . . 7
5.2. Lo que dicen los datos . . . 8
5.3. Cómo calcular la media de cada usuario: encogimiento . . . 9
5.4. Coincidencias y discrepancias . . . 9

6. Las notas bajas **9**

7. Pesos **9**
7.1. Pesar los positivos por la nota 9
7.2. Pesar cada usuario 10

8. Evaluación y split **10**
8.1. Métricas de top-K 10
8.2. ¿Qué es «relevante» en test? 10
8.3. Tipos de split (bloque D, pero afecta aquí) 11

9. Las preguntas **11**

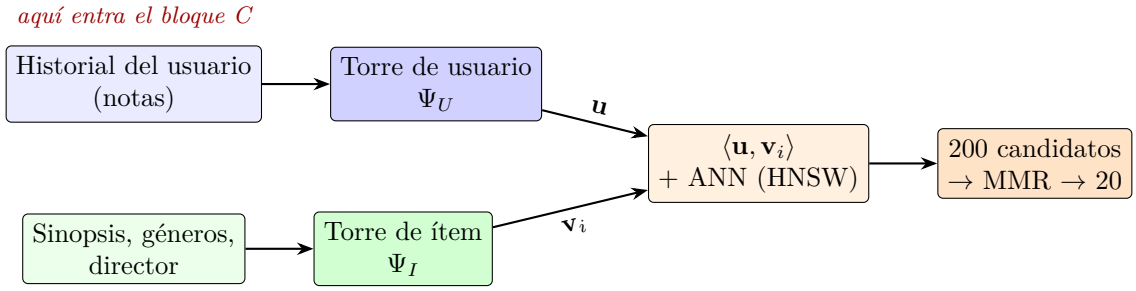
Glosario **14**

1. Dónde estamos y qué se decide aquí

1.1. Lo ya decidido (bloques A y B)

- **Datos**: solo `ml-32m/ratings.csv` (1995–2023). De Kaggle/TMDB solo usamos metadatos.
- **Limpieza**: fuera las películas con menos de 20 ratings (se pierde el 0,86 % de los ratings); quedan 31 725 154 ratings, 200 948 usuarios y 23 335 películas. Duplicados (mismo `tmdbId`) fusionados. Sinopsis para el 99,4 % de las películas.
- **Submuestra de desarrollo**: el 10 % de los usuarios (enteros), estratificada. Todo lo de esta guía está medido ahí; los resultados finales se harán con el conjunto completo.
- Pendiente: B3 (géneros de MovieLens o de TMDB).

1.2. La arquitectura que prometimos en E1



En E1 escribimos, para MovieLens: «*binarización implícita ponderada por* r_{ui} → torre de usuario», y para el entrenamiento: «contrastivo con corrección logQ». El bloque C decide qué quiere decir exactamente eso.

1.3. Qué decide el bloque C

- **Qué aprende el modelo**: ¿predecir la nota o distinguir lo que gusta de lo que no?
- **Qué es un positivo**: ¿a partir de qué nota decimos «le gusta»? ¿Igual para todos los usuarios?
- **Qué hacemos con lo que no gusta** y **cuánto pesa** cada positivo y cada usuario.
- **Cómo se mide** si el modelo acierta, y cómo afecta todo eso a logQ y a los negativos.

Tres bloques que van juntos

El bloque C no se puede cerrar del todo solo. El **D** (cómo partimos train/validación/test) decide qué positivos hay en test y con qué datos se calcula todo lo «por usuario». El **F** (torre de usuario) decide si **u** sale de un identificador o del historial; en el segundo caso, la definición de positivo decide también qué entra en ese historial. Por eso proponemos cerrar **C1, D1 y F1 a la vez**.

2. Los datos que tenemos: notas explícitas

2.1. Explícito frente a implícito

- **Feedback explícito**: el usuario dice cuánto le gustó (MovieLens: de 0,5 a 5 estrellas en pasos de 0,5).

- **Feedback implícito**: solo sabemos que *hizo algo* (vio, clicó, compró). No hay «no me gustó»; lo que no aparece puede ser que no le interese o que no lo conozca. Es lo que tienen YouTube o Spotify, y para lo que se diseñaron la Two-Tower y logQ [4, 11, 1, 12].

Nosotros tenemos explícito, pero lo vamos a usar *como si fuera implícito* («binarizar»): convertir cada nota en «positivo» o «no positivo». La razón es que lo que queremos es **ordenar** un catálogo de 23 335 películas y quedarnos con las 200 mejores, no adivinar la nota exacta.

2.2. Los datos no faltan al azar

Un usuario no puntúa películas al azar: puntúa las que eligió ver, y elige las que espera que le gusten. Por eso hay muchas más notas altas que bajas y una película sin nota *no* es una película que no le gusta. A esto se le llama datos *missing not at random* (MNAR) [9].

Nota	0,5	1,0	1,5	2,0	2,5	3,0	3,5	4,0	4,5	5,0
% ratings	1,7	3,0	1,6	6,3	5,1	18,9	13,3	26,3	9,2	14,5

La nota media es 3,54; el 63 % de las notas son $\geq 3{,}5$ y el 50 % son $\geq 4{,}0$.

2.3. Cada usuario usa la escala a su manera

	p5	p25	p50	p75	p95
Media de cada usuario	2,90	3,42	3,73	4,03	4,46
Desviación típica de cada usuario	0,51	0,73	0,89	1,07	1,37

- El 27,4 % de los usuarios tiene media $\geq 4{,}0$ (para ellos un 4 es «normal») y el 6,9 % tiene media $< 3{,}0$ (para ellos un 3,5 es un elogio).
- 30 usuarios ponen siempre la misma nota; 904 (4,5 %) tienen desviación $< 0{,}5$.
- Los usuarios que puntúan mucho son algo más duros: correlación $-0{,}19$ entre log(nº de ratings) y la media.

2.4. Las medias estrellas no existían antes de 2003

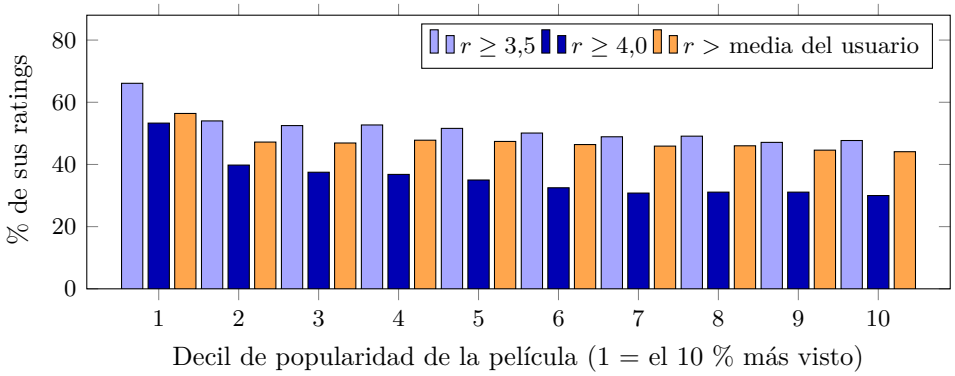
MovieLens solo tenía estrellas enteras hasta 2003 [3] (en nuestros datos el primer 0,5 aparece ese año). El 24,3 % de los ratings es anterior, y el 34,3 % de los usuarios nunca usa medias estrellas. Para todos ellos, «$\geq 3{,}5$» y «$\geq 4{,}0$» son exactamente lo mismo, y el 3 mezcla «estuvo bien» con «regular».

2.5. Mucha gente lo puntúa todo el primer día

- El **62,5 %** de los ratings se hace en las primeras 24 h del usuario, y el **59 %** de los usuarios hace *todas* sus valoraciones ese primer día.
- El 22,3 % de los ratings comparte el segundo exacto con otro del mismo usuario (no hay orden entre ellos).
- El primer día se puntúa más alto: media 3,60 frente a 3,44; ≥ 4 el 53 % frente al 45 %.

Es decir: la mayoría de las notas son *recuerdos* que el usuario introduce de golpe al registrarse (empezando por sus favoritas), no lo que acaba de ver. Esto importa mucho para el bloque D (ver §8).

2.6. Las populares gustan más (de verdad)



El 10 % de películas más vistas reúne el 78,8 % de los ratings, y además allí la tasa de positivos es más alta (53 % de notas ≥ 4 frente a 30–40 % en el resto). Correlación entre log(nº de ratings) y nota media de la película: 0,26. Las populares no solo se ven más: también gustan más. Ninguna corrección «de muestreo» (logQ) va a quitar eso; si queremos más cola larga, es trabajo del MMR.

3. Tres usuarios inventados

Los usaremos durante toda la guía.

Película	**Ana** (generosa)	**Bruno** (exigente)	**Carla** (2001, solo enteras)
Pulp Fiction	—	4,0	5
The Matrix	5,0	3,5	4
Shrek	4,5	3,0	4
Titanic	5,0	1,5	3
Notting Hill	4,0	2,0	—
Batman & Robin	3,5	—	1
Media	4,4	2,8	3,4

- El 3,5 de Ana a *Batman & Robin* es su *peor* nota; el 3,5 de Bruno a *The Matrix* es de las mejores.
- *Titanic* es positivo para Ana, nota baja para Bruno y neutra para Carla.
- Carla no podía poner 3,5: su 3 y su 4 significan cosas más anchas.

4. Cómo aprende una Two-Tower

4.1. Dos torres y un producto escalar

La torre de usuario convierte al usuario en un vector $\mathbf{u} \in \mathbb{R}^{128}$; la torre de ítem convierte cada película en $\mathbf{v}_i \in \mathbb{R}^{128}$. La puntuación es $s(u, i) = \langle \mathbf{u}, \mathbf{v}_i \rangle$. Como los $\mathbf{v}_i$ no dependen del usuario, se calculan una vez para todo el catálogo y se meten en un índice ANN: recomendar es buscar los vectores con mayor producto escalar con $\mathbf{u}$.

Idea clave

La puntuación $s(u,i)$ no tiene por qué ser una nota. Solo importa el **orden**: que lo que gusta quede por encima de lo que no. Por eso podemos «olvidar» la escala de estrellas y quedarnos con positivos.

4.2. Opción 1: regresión de la nota

Se entrena para que $s(u,i) \approx r_{ui}$ minimizando el error cuadrático $\sum(r_{ui} - s(u,i))^2$ sobre los ratings que existen. Así se ganó el Netflix Prize [7]. Problemas para nosotros:

- Solo ve películas que el usuario *eligió*; nunca aprende que una película de nicho que él jamás vería debe quedar abajo. Pero el ANN busca en *todo* el catálogo.
- Un buen RMSE no implica un buen top-K [2].
- No hay negativos que muestrear, así que logQ no tiene sentido. Contradice E1.

4.3. Opción 2: pérdida contrastiva (softmax) con negativos del lote

Para cada par (usuario, película positiva) se pide que el positivo gane a las demás películas:

$$\mathcal{L}(u, i^+) \;=\; -\log \frac{\exp\left(s(u,i^+)/T\right)}{\exp\left(s(u,i^+)/T\right) + \sum_{j \in \mathcal{N}} \exp\left(s(u,j)/T\right)}.$$

T es la *temperatura* (controla lo «afilada» que es la softmax). Lo ideal sería que $\mathcal{N}$ fuera todo el catálogo, pero es carísimo. El truco de [12]: en un lote de B pares (u_k, i_k^+), las películas positivas de los *otros* usuarios del lote hacen de negativos. Sale gratis: una sola multiplicación de matrices $B \times B$.

Ejemplo: un lote de tamaño 3

Lote: (Ana, Matrix), (Bruno, Pulp Fiction), (Carla, Shrek). Para Ana, el positivo es *Matrix* y los negativos son *Pulp Fiction* y *Shrek*. Si $s = 2{,}0$, $1{,}0$ y $0{,}5$ ($T = 1$):

$$\mathcal{L} = -\log \frac{e^2}{e^2 + e^1 + e^{0{,}5}} = -\log \frac{7{,}39}{11{,}76} = 0{,}46.$$

Fíjate: *Shrek* es un positivo de Ana (le puso 4,5) y aquí cuenta como negativo. Es un **falso negativo** (§4.5).

4.4. Sesgo de muestreo y corrección logQ

Los negativos del lote no son películas al azar: una película aparece como negativo tantas veces como aparece como *positivo* en los datos. Si *Matrix* es el positivo del 2 % de los ejemplos y *Shrek* del 0,5 %, Matrix recibe cuatro veces más «empujones hacia abajo» que en una softmax completa. Sin corregir, el modelo infravalora sistemáticamente lo popular por un motivo puramente técnico.

La corrección de [12] resta, solo durante el entrenamiento, el logaritmo de esa probabilidad:

$$s^*(u,j) \;=\; s(u,j) - \log P(j), \qquad P(j) = \text{probabilidad de que } j \text{ salga en un lote.}$$

Con eso, el gradiente es (en esperanza) el de la softmax completa. En inferencia se usa $s(u,j)$ sin corregir.

Lo que logQ hace y lo que no

- logQ **no** es un «castigo a lo popular»: corrige que el lote castigue de más a lo popular. El resultado se parece al de una softmax completa, que refleja la popularidad real.
- En E1 escribimos que logQ «evita que los títulos comerciales acaparen las recomendaciones». Eso es más bien lo contrario de lo que hace: **hay que reformularlo en la E2**. Lo que reduce el peso de lo popular es el MMR, o multiplicar la corrección por un factor $\alpha < 1$ (el τ de la fórmula de E1).
- $P(j)$ debe ser la frecuencia **con la que de verdad se generan** los positivos del entrenamiento. Si cambiamos qué es un positivo (C2) o cómo se muestrean (C4, C5), cambia $P(j)$.

Ejemplo: contar $P(j)$ con todos los ratings o con los positivos

Entre las 300 películas más vistas de la submuestra:

Película	Puesto (todos)	Puesto ($r \geq 4$)	% ≥ 4
Charlie's Angels (2000)	242	913	18
Mission: Impossible II (2000)	253	681	24
12 Angry Men (1957)	254	133	83
Rear Window (1954)	210	115	80

Si contáramos $P(j)$ con todos los ratings, a *Charlie's Angels* le restaríamos demasiado durante el entrenamiento, el modelo lo compensaría subiéndole la puntuación, y en inferencia la recomendaría de más. A los clásicos les pasaría lo contrario. Para las películas con ≥ 100 ratings, $P_{\text{pos}}/P_{\text{todos}}$ va de 0,29 a 1,46 (p5–p95): diferencias de hasta 1,2 en $\log P$.

Si una película no tiene ningún positivo, $\log P = -\infty$: hay que suavizar (sumar 1 a todos los recuentos). Con umbral ≥ 4 hay 3 915 películas así en la submuestra.

4.5. Falsos negativos

Con negativos del lote, el negativo que le toca a un usuario es un positivo suyo el **7,2 %** de las veces de media (p90: 16 %), y algo que ya valoró el 11,1 %. Solución habitual: *enmascarar* (poner $-\infty$ en la softmax) las películas que el usuario ya tiene. Cuesta una máscara por lote.

4.6. Negativos duros

Un *negativo duro* es uno parecido al positivo (difícil de separar). Las notas bajas de un usuario lo son: las eligió porque parecían de su gusto. Usarlas como negativos da más información, pero puede obligar al modelo a separar películas casi iguales. Dato: la probabilidad de que un negativo del lote sea justo una nota baja del usuario es solo del 1,1 %. Si queremos que el modelo vea los «no me gustó», hay que meterlos a propósito.

5. De nota a positivo: umbrales

5.1. Umbral absoluto, relativo o mixto

- **Absoluto**: positivo si $r_{ui} \geq t$ (p. ej. $t = 4$). Igual para todos.
- **Relativo**: positivo si $r_{ui} \geq \bar{r}_u$ (la media del usuario). Quita el sesgo de cada usuario.
- **Mixto**: $r_{ui} \geq \min(4; \bar{r}_u)$ (baja el listón a los exigentes, nunca lo sube por encima de 4) o $r_{ui} \geq \max(3{,}5; \bar{r}_u)$ (lo sube a los generosos).

- **Residuo**: positivo si r_{ui} supera lo esperable para ese usuario y esa película: $r_{ui} - (\mu + b_u + b_i) > 0$, con μ la media global, b_u el sesgo del usuario y b_i el de la película [7]. Mide «le gustó *más de lo esperado*».

Los tres usuarios con cada umbral

Umbral	Ana (4,4)	Bruno (2,8)	Carla (3,4)
$\geq 3{,}5$	las 5 (también su peor)	Pulp, Matrix	Pulp, Matrix, Shrek
$\geq 4{,}0$	4 (no Batman & Robin)	solo Pulp	Pulp, Matrix, Shrek
$\geq$ media	Titanic, Matrix, Shrek (su 4 a Notting Hill no)	Pulp, Matrix, Shrek (su 3 sí)	Pulp, Matrix, Shrek
$\geq$ mín(4; media)	4 (umbral 4,0)	Pulp, Matrix, Shrek (umbral 2,8)	Pulp, Matrix, Shrek
$\geq$ máx(3,5; media)	Titanic, Matrix, Shrek	Pulp, Matrix	Pulp, Matrix, Shrek

Con umbral fijo, Ana aporta muchos positivos flojos y Bruno casi ninguno. Con umbral relativo, el 4 de Ana deja de contar (¿un 4 no es un 4?). El mixto mín$(4; \bar{r}_u)$ deja intacta a Ana y rescata a Bruno.

5.2. Lo que dicen los datos

Umbral	% pos.	Pos./usuario (med.; p10)	Usuarios sin pos.	Sin pos. en test*	Películas ≥ 20 pos.	% pos. en top 10 %
$r \geq 3{,}0$	82,2	63; 21	6	47	8 130	80,9
$r \geq 3{,}5$	63,4	49; 15	17	249	6 911	82,2
$r \geq 4{,}0$	50,0	42; 13	26	374	5 805	83,8
$r \geq 4{,}5$	23,7	20; 5	229	2 490	3 689	85,9
$r \geq \bar{r}_u$	54,7	40; 13	0	234	6 581	81,6
$r \geq$ mín$(4; \bar{r}_u)$	59,4	46; 15	0	151	6 777	81,9
$r \geq$ máx$(3{,}5; \bar{r}_u)$	51,7	39; 12	17	350	6 229	82,3
residuo > 0	53,4	37; 13	1	195	7 017	79,5

* «Test» provisional: el último 20 % de los ratings de cada usuario por fecha. Referencia: con todos los ratings, el top 10 % de películas reúne el 78,8 %.

Cómo leerla:

- **Usuarios sin positivos en test** salen de las métricas (no se puede medir si aciertas si no hay nada que acertar). Cuantos menos, mejor.
- **Películas con ≥ 20 positivos**: las que el modelo puede aprender bien por la parte colaborativa; el resto dependerá sobre todo de la sinopsis y los géneros.
- **% en top 10 %**: cuanto más estricto el umbral, más se concentran los positivos en las populares.

5.3. Cómo calcular la media de cada usuario: encogimiento

Con pocos ratings, la media es ruidosa. La media *encogida* tira hacia la media global $\mu = 3{,}54$:

$$\tilde{r}_u = \frac{\sum_i r_{ui} + \lambda\,\mu}{n_u + \lambda}, \qquad \lambda \approx 10.$$

Ejemplo

Un usuario nuevo de Letterboxd ha puntuado 3 películas: 5, 5 y 4,5. Media cruda: 4,83 (umbral mixto mín$(4; 4{,}83) = 4$). Media encogida: $(14{,}5 + 35{,}4)/13 = 3{,}84$ (umbral 3,84). Con 300 ratings, las dos medias casi coinciden.

Lo mismo se usa para los sesgos b_u y b_i (en la submuestra, b_u va de $-0{,}56$ a 0,77 y b_i de $-0{,}49$ a 0,26, p5–p95).

Fuga de información

$\bar{r}_u$ se tiene que calcular **solo con los ratings de entrenamiento**. Si se calcula con todos, el umbral de train «sabe» las notas de test, y las métricas salen mejores de lo que son. Esto vale para cualquier cosa «por usuario» o «por película» (medias, sesgos, $P(j)$ de logQ).

5.4. Coincidencias y discrepancias

- El 8,3 % de los ratings son ≥ 4 pero tienen residuo ≤ 0 (gustó, pero menos de lo esperable: un 4 a *Cadena perpetua*).
- El 9,3 % son < 4 pero están por encima de la media del usuario (los «3,5 de Bruno»).

6. Las notas bajas

Definición	% ratings	% usuarios con ≥ 1	Mediana por usuario	% en top 10 %
$r \leq 2{,}0$	12,7	85,6	5	68,8
$r \leq 2{,}5$	17,8	90,2	7	69,1
$r \leq \bar{r}_u - 1$	13,4	95,9	8	71,8

Hay tres formas de usarlas:

1. **Ignorarlas** (como hace YouTube [1]): solo positivos y negativos del lote.
2. **Negativos duros en la pérdida**: cada positivo compite además con 1–5 notas bajas del usuario. Ojo: logQ no se aplica a ellas porque no salen del mismo muestreo.
3. **Anti-preferencias en la torre de usuario**: **u** se calcula con «lo que le gustó» y, por separado, «lo que no». No cambia la pérdida; es una decisión del bloque F.

Entre «nota baja» y el umbral de positivo queda una **zona neutra** (p. ej. $2{,}5 < r < 4$): hay que decidir si esas películas entran en el historial que lee la torre de usuario.

7. Pesos

7.1. Pesar los positivos por la nota

Con pesos, la pérdida pasa a ser $\sum w_{ui}\,\mathcal{L}(u, i)$. Ejemplos, con umbral t: lineal $w = 1 + (r - t)$; exponencial $w = 2^{r-t}$; relativo $w = 1 + \max(0, r - \bar{r}_u)$. Con el lineal y $t = 4$, el 5 de Ana a *Titanic* pesa 2 y su 4 a *Notting Hill* pesa 1.

Tamaño efectivo de muestra (ESS) [6]: cuánto «vale» una muestra con pesos frente a una sin pesos,

$$\mathrm{ESS} = \frac{(\sum w)^2}{N \sum w^2} \in (0, 1].$$

Con 4 positivos de pesos 1, 1, 1 y 3: ESS $= 36/48 = 0{,}75$, es decir, es como tener 3 ejemplos sin pesos. Pesos muy desiguales ⇒ gradiente más ruidoso.

Pesos (positivos $r \geq 3{,}5$)	ESS (%)	% del peso total en…		
		notas 5	top 10 % películas	top 10 % usuarios
$w = 1$	100	22,9	82,2	42,6
lineal $w = r - 2{,}5$	91,2	33,7	83,3	39,8
exponencial $w = 2^{r-3{,}5}$	87,3	37,3	83,5	39,3
relativo $1 + \max(0, r - \bar{r}_u)$	90,2	31,2	82,5	43,9
$w = 1/\sqrt{n_u}$	73,4	27,0	86,6	22,0
$w = (r - 2{,}5)/\sqrt{n_u}$	64,0	38,4	87,3	20,0

Dos maneras de aplicar un peso: **multiplicar la pérdida** (no cambia qué sale en los lotes, logQ intacto) o **muestrear** más a menudo los ejemplos que pesan más (cambia qué sale en los lotes, y entonces $P(j)$ de logQ hay que contarlo con esos pesos).

7.2. Pesar cada usuario

El 10 % de usuarios más activos aporta el **46,6 %** de los ratings. Si cada rating pesa igual, el modelo se optimiza para ellos; pero las métricas promedian *por usuario*, y los usuarios de Letterboxd tendrán pocos ratings. Opciones: $w = 1/\sqrt{n_u}$ (compromiso), o generar los ejemplos por usuario con un tope por época, como YouTube [1]: «dado su historial, ¿cuál es el siguiente positivo?». Lo segundo solo existe si la torre de usuario lee el historial (bloque F).

8. Evaluación y split

8.1. Métricas de top-K

Para cada usuario de test se generan K recomendaciones (excluyendo lo que ya vio en train) y se comparan con sus películas relevantes de test R_u.

$$\mathrm{Recall@}K = \frac{|\text{top-}K \cap R_u|}{|R_u|}, \qquad \mathrm{NDCG@}K = \frac{\sum_{k=1}^{K} \frac{\mathbb{1}[\text{pos. } k \in R_u]}{\log_2(k+1)}}{\text{lo mismo con el orden ideal}} \ [5].$$

Ejemplo

Ana tiene 3 relevantes en test y le recomendamos 5 películas; aciertan la 1.ª y la 4.ª. Recall@5 $= 2/3 = 0{,}67$. DCG $= 1/\log_2 2 + 1/\log_2 5 = 1 + 0{,}43 = 1{,}43$; ideal (3 aciertos arriba): $1 + 0{,}63 + 0{,}5 = 2{,}13$; NDCG@5 $= 0{,}67$. NDCG premia acertar *arriba*.

Las métricas se calculan sobre el catálogo completo, no sobre 100 negativos al azar: las métricas «muestreadas» pueden invertir el orden entre modelos [8]. Además conviene medir **cobertura** (qué fracción del catálogo llega a recomendarse) y **novedad** (lo poco populares que son las recomendaciones), que es lo que el MMR debe mejorar.

8.2. ¿Qué es «relevante» en test?

Si «relevante» usa la misma definición que el entrenamiento, al cambiar el umbral cambia también la vara de medir y dos modelos dejan de ser comparables. Una solución: medir con las dos (la de train y una fija, $r \geq 4$, comparable con la literatura).

8.3. Tipos de split (bloque D, pero afecta aquí)

- **Aleatorio**: ratings al azar a test. Fácil, pero el modelo «ve el futuro».
- **Último(s) de cada usuario** (*leave-last-out* o último 20 %): realista, pero en el 59 % de los usuarios todo pasa el mismo día y con empates de segundo: el «orden» es casi arbitrario.
- **Temporal global**: todo lo anterior a una fecha a train, lo posterior a test. Sin fugas temporales, pero muchos usuarios de test no tienen nada en train (eso es justo el caso Letterboxd).
- **Por usuarios**: unos usuarios en train y otros en test, a los que se les da una parte de su historial y se predice el resto. Encaja con una torre de usuario basada en historial.

Comparativa de estrategias en [10]. Pase lo que pase, todo lo que se calcula «por usuario» o «por película» (§5.3) sale solo de train.

9. Las preguntas

$\star$ = mi recomendación, no una decisión. «D» / «F» = depende de ese bloque.

C1. Tipo de señal

D1, F1

¿Qué predice el modelo?

a) Regresión de la nota (error cuadrático). Sin negativos ni logQ; contradice E1.

b) $\star$ Positivo / no positivo con pérdida contrastiva, negativos del lote y logQ (lo de E1).

c) Como b, pero positivo relativo a la media del usuario (en la práctica, una opción de C2).

d) Las dos cosas: contrastiva + una salida que predice la nota.

C2. Umbral de positivo

D (media solo con train)

¿A partir de qué nota «le gusta»?

a) $r \geq 3{,}5$ (63,4 % positivos; 17 usuarios sin ninguno).

b) $r \geq 4{,}0$ (50,0 %; 26 sin ninguno; 374 sin positivos en test). El estándar de los papers.

c) $r \geq$ media del usuario (54,7 %; 0 sin positivos; 234 sin positivos en test).

d) $\star$ $r \geq \text{mín}(4{,}0;\ \text{media})$ (59,4 %; 0; 151). Con b como ablación.

e) $r \geq \text{máx}(3{,}5;\ \text{media})$ (51,7 %; 17; 350).

f) Residuo > 0. Desaconsejado como objetivo.

C3. Notas bajas

F

¿Qué hacemos con lo que no le gustó?

a) $\star$ (primera versión) Tirarlas.

b) Negativos duros en la pérdida.

c) Anti-preferencias en la torre de usuario.

d) a + c.

C4. Pesar los positivos por la nota C2

¿Un 5 vale más que un 4?

a) No: todos pesan 1.

b) ⋆ Lineal $w = 1 + (r - t)$, multiplicando la pérdida (ESS 91 %; es lo que prometimos en E1).

c) Exponencial $w = 2^{r-t}$.

d) Relativo $w = 1 + \max(0, r - \bar{r}_u)$.

C5. Peso de cada usuario F1, C13

¿Cuentan igual los que puntúan 20 y 2 000 películas?

a) Nada (el top 10 % de usuarios aporta el 46,6 %).

b) $w = 1/\sqrt{n_u}$.

c) ⋆ Ejemplos por usuario con tope por época (o b si F no usa historial).

C6. Media del usuario D

¿Cómo se calcula (siempre solo con train)?

a) Media normal.

b) ⋆ Media encogida, $\lambda \approx 10$.

c) Mediana.

C7. Relevante en test C2, D

¿Qué cuenta como acierto?

a) Lo mismo que en train.

b) Siempre $r \geq 4$.

c) ⋆ Las dos.

C8. Usuarios sin positivos D

¿Qué hacemos con ellos?

a) Quitarlos del todo.

b) ⋆ Mantenerlos en train y dejarlos fuera de las métricas.

C9. Valoraciones en bloque el primer día D

¿Las tratamos distinto?

a) ⋆ Nada en C; se decide en D.

b) Solo como historial, nunca como objetivo.

c) Con menos peso.

C10. Usuarios sin medias estrellas — C2

¿Ajustamos la escala?

a) ⋆ Nada (si C2 es c o d, la media ya lo absorbe).

b) Umbral distinto según la época.

C11. Ratings antes del estreno —

218 en total.

a) Mantenerlos.

b) ⋆ Quitarlos.

c) Solo como historial.

C12. Re-ratings —

MovieLens no tiene; Letterboxd sí.

a) ⋆ Quedarse con la última nota (como en B1b).

b) La media de las notas.

C13. logQ — C2, C4, C5

¿Con qué se cuenta $P(j)$?

a) Con todos los ratings.

b) Con los positivos de train, más suavizado +1.

c) ⋆ Como b, pero contando como se generan de verdad los ejemplos de entrenamiento.

C14. Falsos negativos del lote — C3

¿Enmascaramos?

a) Ignorarlos (7,2 % de los negativos son positivos del propio usuario).

b) Enmascarar los positivos del usuario.

c) ⋆ Enmascarar todo lo que el usuario ya valoró.

Resumen de dependencias

Pregunta	Depende de D porque…	Depende de F porque…
C1	el split decide qué positivos hay en test	si **u** sale del historial, C decide qué entra en él
C2, C6	la media del usuario solo con train	—
C3	—	la opción c es una entrada de la torre de usuario
C5	—	«ejemplos por usuario» solo existe si F usa historial
C7, C8	definen el test	—
C9	en el 59 % de los usuarios no hay orden temporal real	—
C13	$P(j)$ solo con train	depende del generador de ejemplos (C5)

Orden propuesto: C1 + D1 + F1 juntas; luego C2, C4 y C6; C3 y C5 con F; C7–C9 con D; C13 y C14 al escribir la pérdida.

Glosario

ANN / HNSW Búsqueda aproximada de vecinos más cercanos; HNSW es el índice en grafo que usaremos.

Binarizar Convertir cada nota en positivo / no positivo.

ESS Tamaño efectivo de muestra con pesos.

Falso negativo Un positivo del usuario usado como negativo.

Fuga (leakage) Usar en entrenamiento información que solo existe en test.

In-batch Usar los positivos de los demás ejemplos del lote como negativos.

logQ Restar $\log P(j)$ para corregir el sesgo de muestreo de los negativos.

MNAR Datos que faltan por un motivo relacionado con su valor (no se puntúa lo que no se espera que guste).

MMR Reordenación que equilibra relevancia y variedad.

NDCG / Recall@K Métricas de top-K (§8).

Residuo Nota menos lo esperable: $r_{ui} - (\mu + b_u + b_i)$.

Temperatura T Divide las puntuaciones en la softmax; más baja, más «afilada».

Referencias

[1] P. Covington, J. Adams, E. Sargin. *Deep Neural Networks for YouTube Recommendations*. RecSys, 2016.

[2] P. Cremonesi, Y. Koren, R. Turrin. *Performance of Recommender Algorithms on Top-N Recommendation Tasks*. RecSys, 2010.

[3] F. M. Harper, J. A. Konstan. *The MovieLens Datasets: History and Context*. ACM TiiS, 2015.

[4] Y. Hu, Y. Koren, C. Volinsky. *Collaborative Filtering for Implicit Feedback Datasets*. ICDM, 2008.

[5] K. Järvelin, J. Kekäläinen. *Cumulated Gain-Based Evaluation of IR Techniques*. ACM TOIS, 2002.

[6] L. Kish. *Survey Sampling*. Wiley, 1965.

[7] Y. Koren, R. Bell, C. Volinsky. *Matrix Factorization Techniques for Recommender Systems*. IEEE Computer, 2009.

[8] W. Krichene, S. Rendle. *On Sampled Metrics for Item Recommendation*. KDD, 2020.

[9] B. M. Marlin, R. S. Zemel. *Collaborative Prediction and Ranking with Non-Random Missing Data*. RecSys, 2009.

[10] Z. Meng, R. McCreadie, C. Macdonald, I. Ounis. *Exploring Data Splitting Strategies for the Evaluation of Recommendation Models*. RecSys, 2020.

[11] S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme. *BPR: Bayesian Personalized Ranking from Implicit Feedback*. UAI, 2009.

[12] X. Yi, J. Yang, L. Hong, D. Z. Cheng, L. Heldt, A. Kumthekar, Z. Zhao, L. Wei, E. Chi. *Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations*. RecSys, 2019.